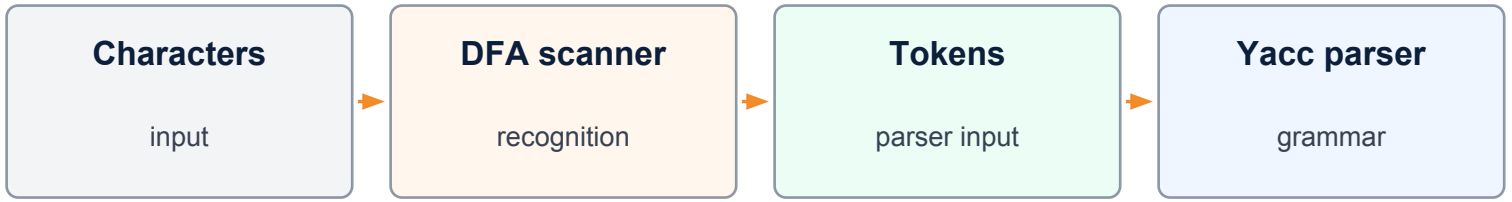
Lexical Analysis

Aho–Sethi–Ullman style concepts with many automata examples

For UG CSE • includes Lex/Flex and Yacc/Bison

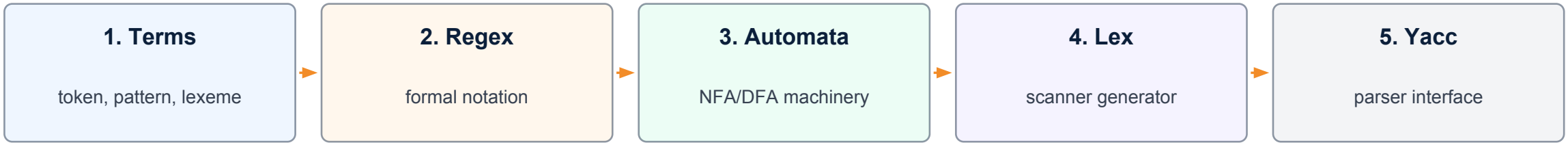


40 slides

regex • NFA • DFA • keyword automata • Lex • Yacc

Learning map

From character streams to parser-ready token streams



Key outcome

Students should be able to design scanners as deterministic finite automata and then implement them using Lex/Flex.

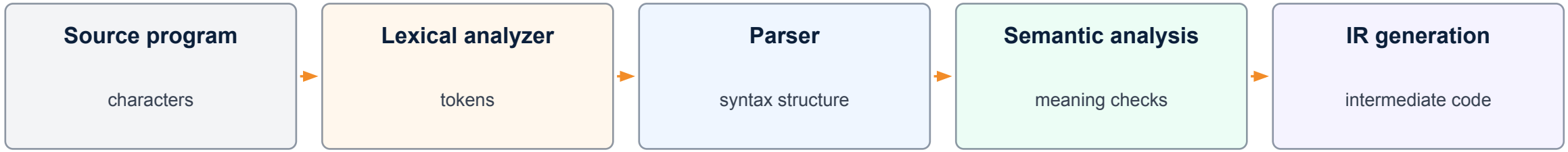
Automata emphasis

This revised deck includes separate automata for keywords, identifiers, operators, numbers, comments, and whitespace.

- Use longest-match and rule-priority principles correctly.
- Recognize why keywords can be handled using tries or reserved-word lookup.
- Connect scanner output with Yacc token declarations and yylval attributes.

Where lexical analysis fits

The lexical analyzer is the first phase of the front end



Scanner job

Hide messy character-level details and return a clean sequence of token classes and attributes.

Parser job

Use grammar productions over token names; it should not handle spaces, comments or raw digits directly.

```
while (x < 10) x = x + 1;

→ WHILE LPAREN ID LT NUM RPAREN ID ASSIGN ID PLUS NUM SEMI
```

Tokens, patterns, lexemes

Three basic units in scanner design

Term	Meaning	Example
Token	Abstract class returned to parser	IF, ID, NUM, PLUS
Pattern	Rule describing valid lexemes	letter(letter|digit)*
Lexeme	Actual text matched in source	count, 42, if
Attribute	Extra value attached to token	symbol table pointer

Memory rule

Token = category; lexeme = actual characters; pattern = description of allowed characters.

Regular expressions

Token patterns are normally specified by regular expressions

```
letter      [A-Za-z_]
digit       [0-9]
id          letter (letter|digit)*
integer     digit+
real        digit+ \. digit+
relop       <= | >= | == | != | < | >
```

- Concatenation: ab means a followed by b.
- Choice: a|b means either a or b.
- Closure: a* means zero or more a.
- Positive closure: a+ means one or more a.
- Character classes make scanner specifications compact.

Bridge to automata

Every regular expression can be implemented by a finite automaton.

From regular expression to automaton

Scanner generators automate this path

NFA advantage

Simple to build mechanically from regex using small fragments.

DFA advantage

At run time, each input character leads to exactly one next state.

Lex/Flex role

Build the automata and emit C scanner code that implements the DFA.

lex specification → generated scanner → yylex() returns tokens

Finite automata notation

States, transitions and accepting states

State

A memory position of the recognizer. q0 is usually the start state.

Transition

A labeled arrow tells what state to go to after reading a character.

Final state

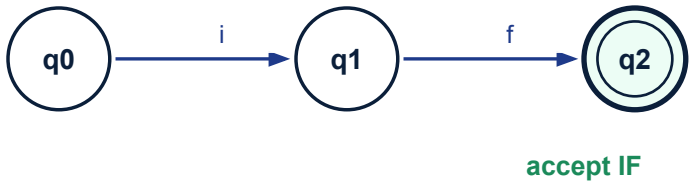
A double circle indicates that a token pattern has been matched.

Recognition

If the automaton stops in a final state after consuming a lexeme, the token is accepted.

Keyword automaton: IF

A small DFA for a two-letter reserved word



Accepting IF

After reading i followed by f, the scanner can accept token IF.

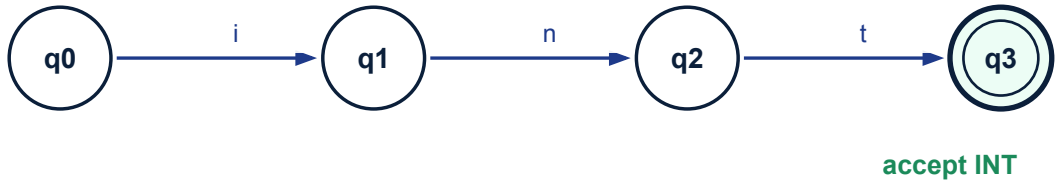
But be careful

The input ifx should not be tokenized as IF then ID(x). Longest match makes it ID(ifx).

```
if   → IF
ifx  → ID("ifx")   // not IF ID
```

Keyword automaton: INT

Linear DFA for the keyword int



Keyword recognition

A reserved-word DFA recognizes exact character sequences.

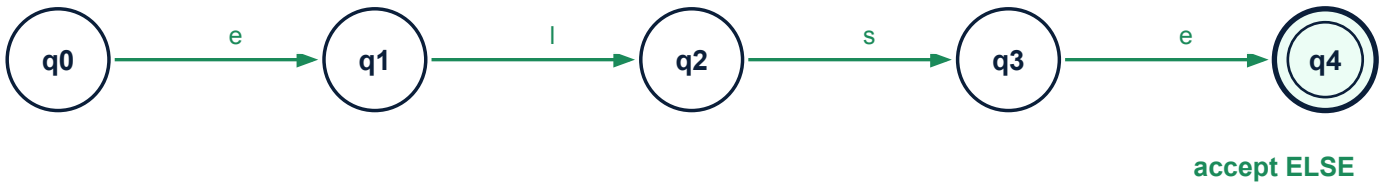
Identifier interaction

integer, int2 and interval are identifiers unless the language says otherwise.

```
int     → INT
integer → ID("integer")
int2    → ID("int2")
```

Keyword automaton: ELSE

DFA path for a four-letter keyword



Common use

ELSE is a reserved word that the parser uses in conditional grammar rules.

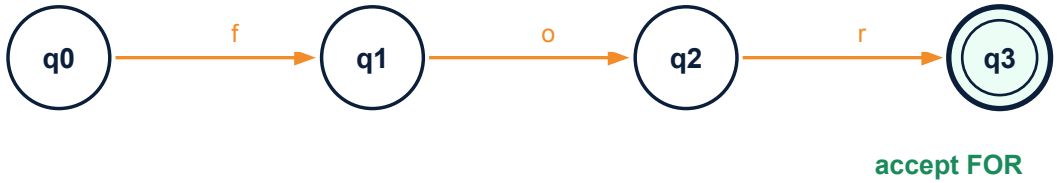
Longest match

elsewhere must be one identifier, not ELSE + ID(where).

```
else      → ELSE
elsewhere → ID("elsewhere")
```

Keyword automaton: FOR

DFA path for a looping keyword



Token use

FOR is consumed by parser productions for loop statements.

Rejecting variants

format, fork, and for2 are not FOR tokens under C-like lexical rules.

```
for    → FOR
fork   → ID("fork")
format → ID("format")
```

Keyword automaton: WHILE

DFA path for while

Longer keyword

The same idea scales: one state per matched prefix.

Prefix property

If a keyword is a prefix of an identifier, longest-match prevents premature acceptance.

```
while  → WHILE
while1 → ID("while1")
```

Keyword automaton: RETURN

DFA path for return

accept RETURN

Reserved-word token

RETURN is usually a token name, not an identifier.

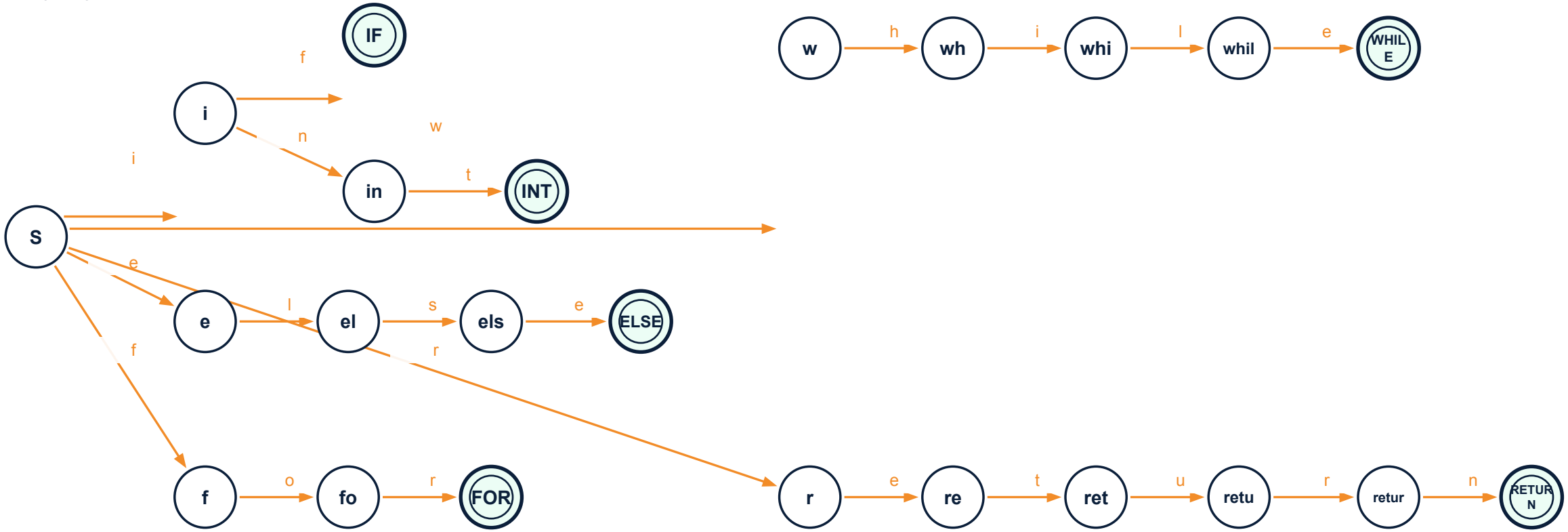
Lookahead needed

The scanner must know the next character is not a letter/digit before finalizing RETURN.

```
return;      → RETURN SEMI
returnValue; → ID("returnValue") SEMI
```

Combined keyword trie automaton

Many keywords can share prefix states



Trie idea

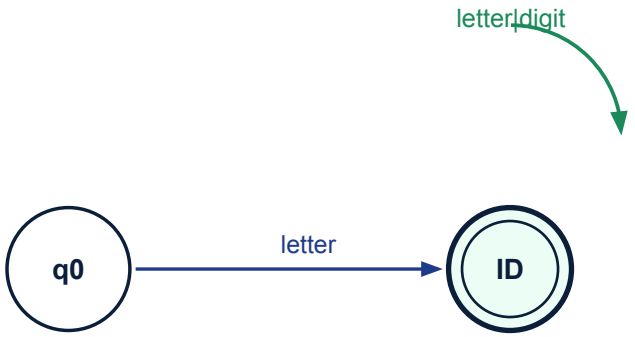
Store shared prefixes once. This is efficient for exact keyword recognition.

Implementation note

Many real scanners instead recognize ID first, then check a reserved-word table.

Identifier DFA

letter followed by letters or digits



Regular expression

ID = letter (letter | digit)*

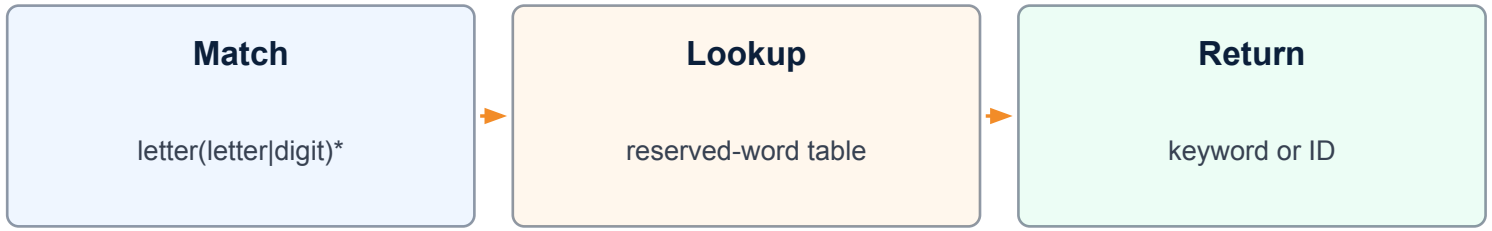
Reserved word handling

After accepting ID, check if the lexeme is in the keyword table. If yes, return keyword token.

```
lexeme = "while"  → keyword table says WHILE
lexeme = "while1" → not a keyword → ID
```

Identifier vs keyword: practical scanner design

Recognize ID broadly, then refine by table lookup



Lexeme	Lookup result	Returned token
if	reserved	IF
int	reserved	INT
count	not reserved	ID(count)
while2	not reserved	ID(while2)

Why this is common

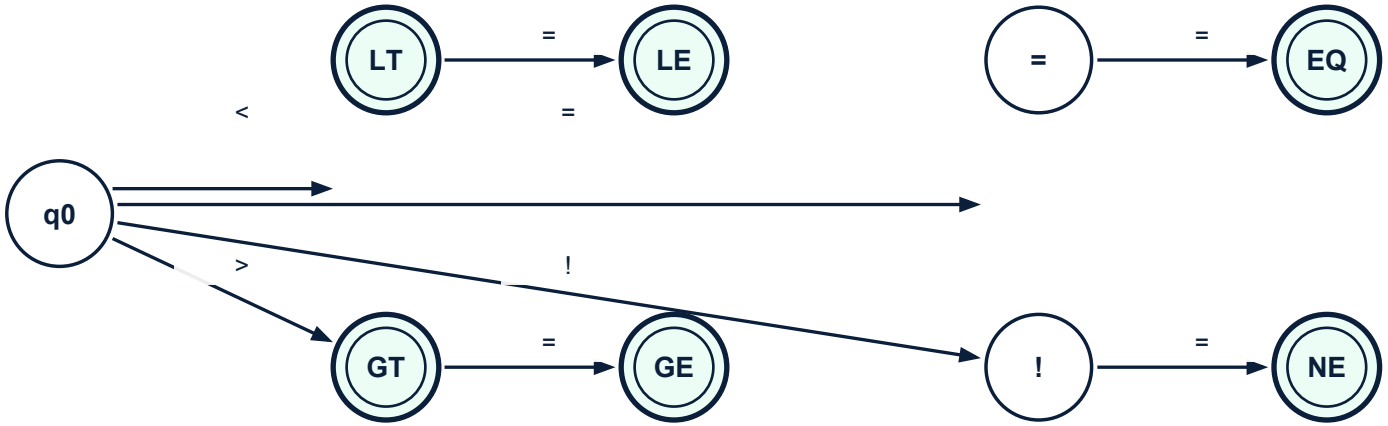
It avoids separate rules for every keyword and handles identifier prefix cases cleanly.

Where automata still matter

The ID rule itself is still implemented as a DFA.

DFA for relational operators

Longest-match is essential for <, <=, >, >=, ==, !=



Fallback needed

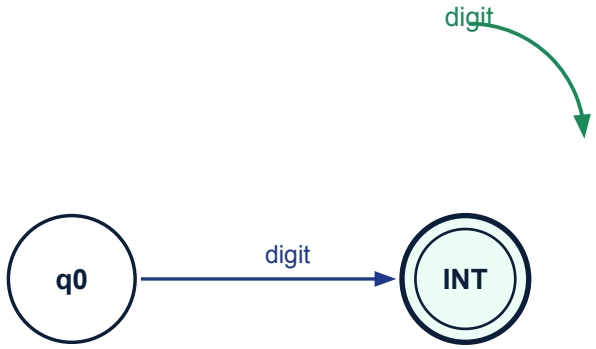
If q=LT sees a non-= next character, retract that character and return LT.

Invalid case

A lone ! may be lexical error in C-like languages unless unary NOT is allowed.

DFA for integer numbers

One or more digits



Regex

digit+

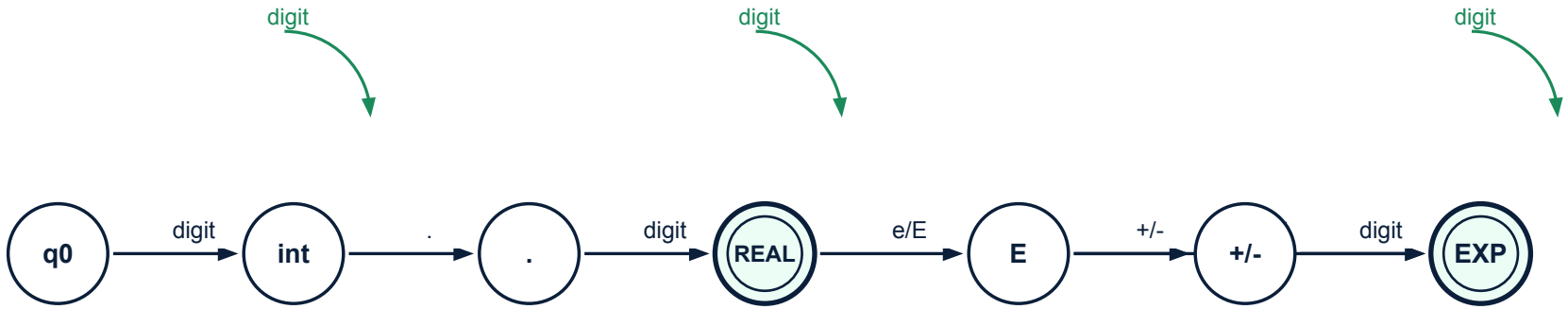
Token attribute

The scanner can compute numeric value while reading digits.

```
value = 0
while isdigit(c):
    value = 10*value + (c - '0')
```

DFA for real numbers

Integer part, decimal point, fraction and optional exponent



Examples accepted

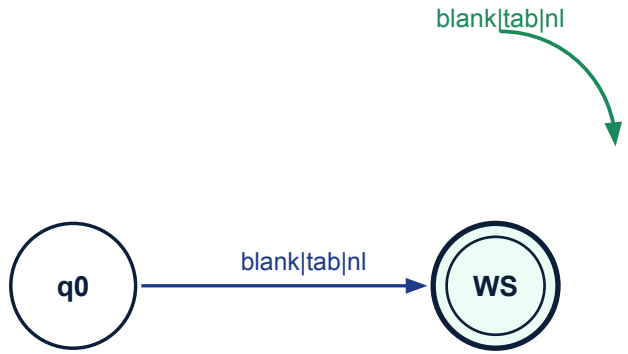
3.14, 0.5, 12.0e-3, 7.2E8

Design choice

Whether 10. is legal depends on the language specification.

DFA for whitespace

Whitespace is recognized and usually skipped



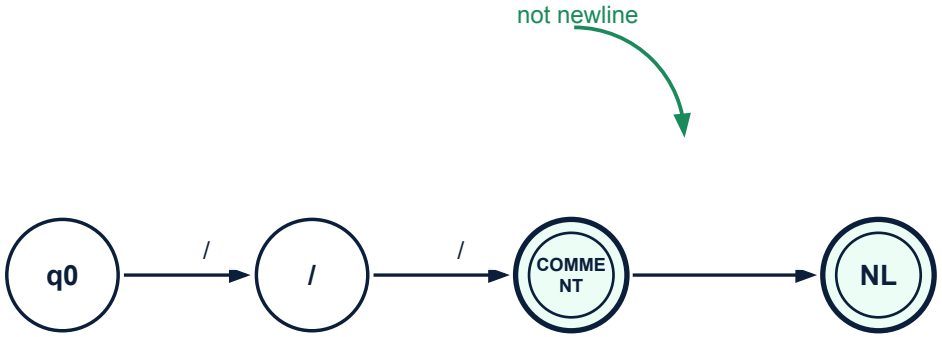
Action

Do not return a token. Update line/column counters and continue scanning.

```
[
]+     { /* skip, update location */ }
```

DFA for single-line comments

Recognize // until newline



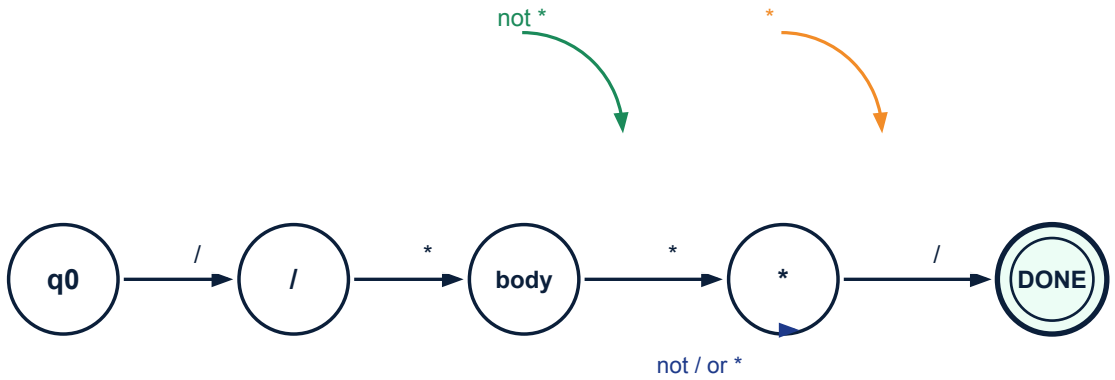
Action

Skip the entire comment and resume scanning at the next line.

```
"//"[^
]*   { /* skip comment */ }
```

DFA for block comments

Recognize /* ... */ carefully



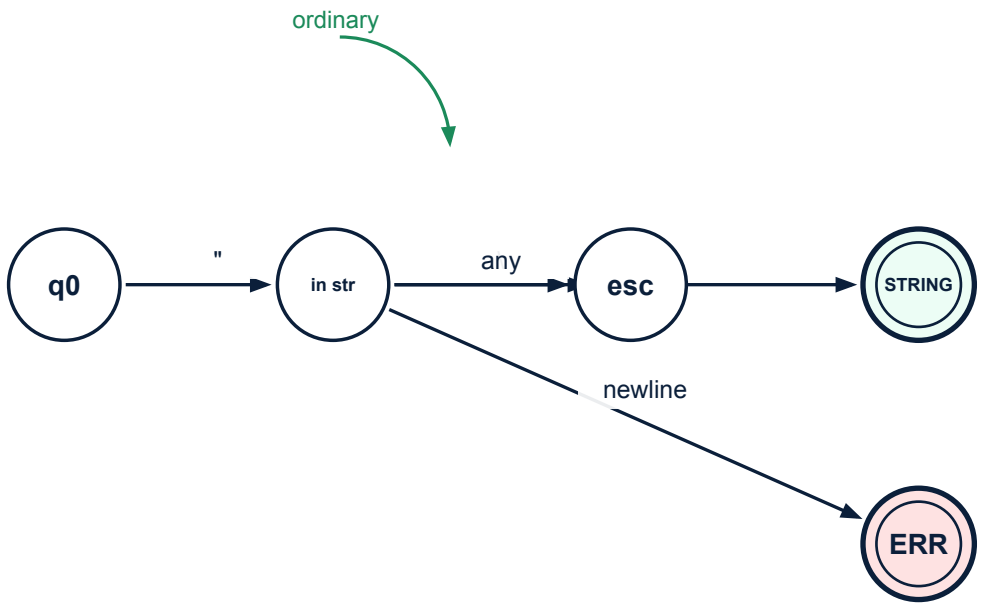
Lexical error

End-of-file before */ gives an unterminated comment error.

```
"/*"([^*]|*+[^*/])*"*/"    { /* skip */ }
```

DFA for string literals

Strings require escapes and termination checks



Action on accept

Return STRING with attribute pointing to literal text after unescaping.

Action on error

Report unterminated string and recover at newline or safe delimiter.

Longest-match rule

Choose the longest prefix accepted by any token pattern

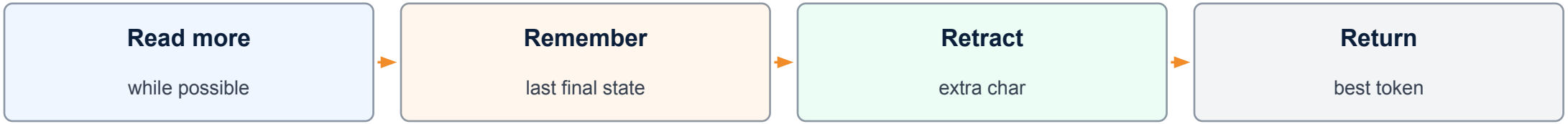
```
input:  count>=10

Correct tokens:
ID(count)  GE(>=)  NUM(10)

Wrong tokens if greedy rule ignored:
ID(count)  GT(>)  ASSIGN(=)  NUM(10)
```

Rule

Scan forward until no transition is possible, then return the token associated with the last accepting state.

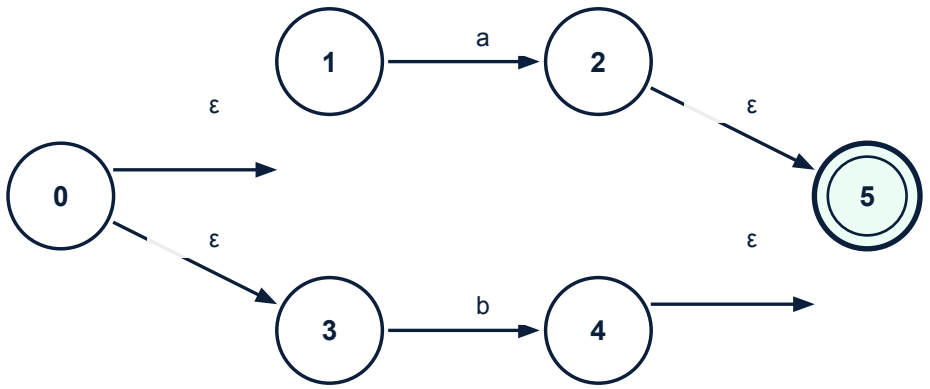
Tie-breaker

If two rules match the same longest lexeme, Lex/Flex chooses the earlier rule.



NFA for a|b

Nondeterminism is convenient for construction



Meaning

Accept either a or b. ε transitions consume no input character.

Use in construction

Regex operators can be converted to small NFA fragments and then combined.

NFA for concatenation ab

Concatenation chains automata in sequence

Recognition

The machine accepts only if a is followed by b.

Scanner link

Multi-character tokens such as <= or == are just concatenations with alternatives.

```
regex:  ab
accepted: ab
rejected: a, b, ba, abc
```

NFA for a*

Closure allows zero or more repetitions

Meaning

Accept empty string, a, aa, aaa, and so on.

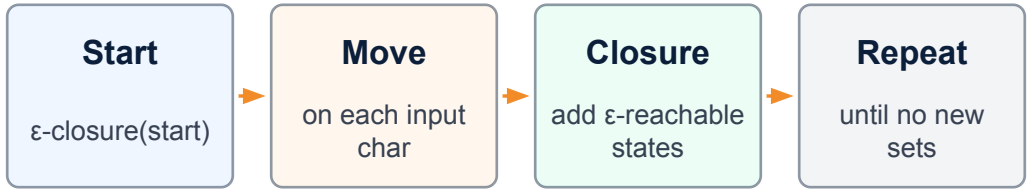
Scanner link

The loop in identifier and number DFAs comes from closure or positive closure.

Subset construction idea

Convert NFA state sets into DFA states

DFA state	NFA-state set	On a	On b
A	{0,1,3}	B	C
B	{2,5}	—	—
C	{4,5}	—	—



Practical result

A DFA state may represent many NFA states. At run time, the scanner only follows one DFA transition per character.

Table-driven DFA scanner

The scanner can be implemented as a transition table

state	letter	digit	other	accept
0	1	2	error	—
1	1	1	retract	ID
2	error	2	retract	NUM

```
state = 0
while true:
    c = nextChar()
    state = T[state,c]
    if state has no transition:
        return token of lastAcceptingState
```

Generated scanners

Lex/Flex often compiles token regexes into such DFA tables plus semantic actions.

Lex/Flex file structure

Definitions, rules, user code

```
%{
#include "parser.tab.h"
%}

digit   [0-9]
letter  [A-Za-z_]

%%
"if"        return IF;
{letter}({letter}|{digit})*  return ID;
{digit}+    return NUM;
[
]+     ;
.           return yytext[0];
%%
int yywrap(void){ return 1; }
```

Section 1

Definitions, C declarations and named regular definitions.

Section 2

Pattern-action rules. This is where automata are generated.

Section 3

Auxiliary C functions used by scanner actions.

Lex rules for keywords

Explicit rules rely on longest-match and rule order

```
%%
"if"        return IF;
"int"       return INT;
"else"      return ELSE;
"for"       return FOR;
"while"     return WHILE;
"return"    return RETURN;
[A-Za-z_][A-Za-z0-9_]*  return ID;
%%
```

Why keyword rules before ID?

When a keyword and ID have the same matched length, the earlier Lex rule wins.

Why ID still handles ifx?

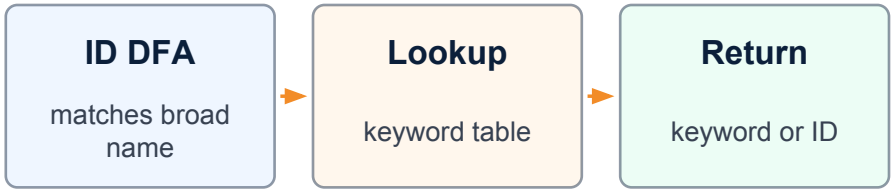
ifx is a longer match for ID than the keyword if. Longest-match wins before rule-order tie-breaking.

```
if   → IF
ifx  → ID
int  → INT
int2 → ID
```

Lex rules using reserved-word lookup

A compact alternative for many keywords

```
%{
int keyword(const char* s);
%}

id   [A-Za-z_][A-Za-z0-9_]*

%%
{id}  {
        int tok = keyword(yytext);
        if(tok) return tok;
        yylval.name = strdup(yytext);
        return ID;
      }
%%
```



Good for large languages

Hundreds of reserved words can be kept in a hash table without adding many separate rules.

Lex actions for attributes

Token values are passed through yylval

```
%{
#include "parser.tab.h"
%}

%%
[0-9]+      { yylval.ival = atoi(yytext); return NUM; }
[A-Za-z_][A-Za-z0-9_]* {
              yylval.sval = strdup(yytext);
              return ID;
            }
"+"         return PLUS;
"="         return ASSIGN;
%%
```

Token name

NUM, ID, PLUS, ASSIGN are grammar-visible token classes.

Attribute

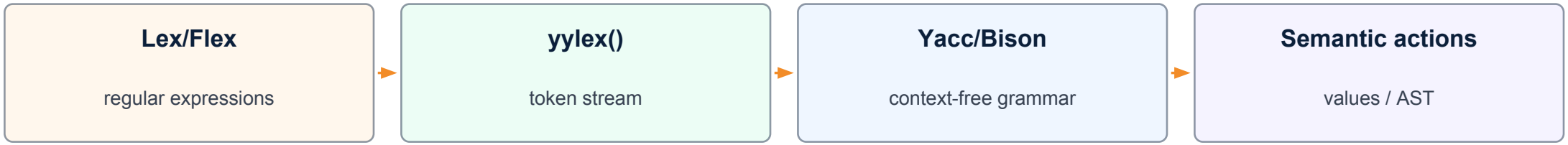
NUM carries integer value; ID carries string or symbol-table pointer.

Parser dependency

The Yacc file must declare compatible semantic-value types.

Yacc/Bison overview

Yacc consumes the tokens returned by Lex



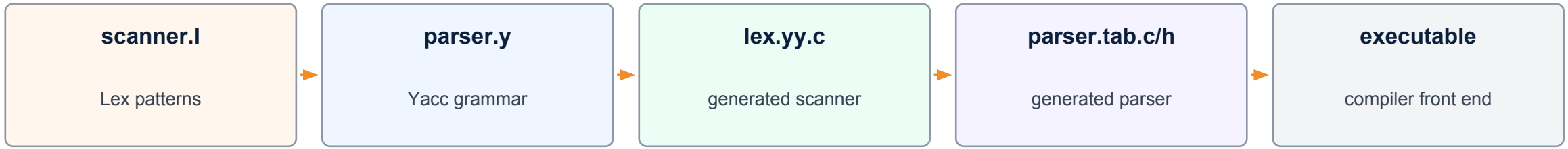

```
%token IF ELSE WHILE RETURN ID NUM
%token PLUS ASSIGN LT GE

%%
stmt : IF '(' expr ')' stmt
     | WHILE '(' expr ')' stmt
     | RETURN expr ';'
     ;
%%
```

Scanner-parser contract

Lex must return exactly the token names that Yacc declares and grammar rules expect.

Lex and Yacc build flow

Typical compilation commands




```
bison -d parser.y      # produces parser.tab.c and parser.tab.h
flex scanner.l         # produces lex.yy.c
gcc parser.tab.c lex.yy.c -lfl -o parser
```

Header dependency

scanner.l includes parser.tab.h so token codes match the parser.

Entry point

The parser calls yylex() whenever it needs the next token.

Mini calculator scanner

Lex rules for numbers and operators

```
%{
#include "calc.tab.h"
%}

%%
[0-9]+      { yylval.ival = atoi(yytext); return NUM; }
"+"         return PLUS;
"-"         return MINUS;
"*"         return MUL;
"/"         return DIV;
"("         return LPAREN;
")"         return RPAREN;
[
]+      ;
.           { return yytext[0]; }
%%
```

Automata inside

The [0-9]+ rule compiles to the integer DFA shown earlier.

Punctuation tokens

Single-character tokens may be returned directly or using named tokens.

Whitespace

Whitespace rule has an empty action, so no token is returned.

Mini calculator parser

Yacc grammar consumes scanner tokens

```
%token <ival> NUM
%token PLUS MINUS MUL DIV LPAREN RPAREN
%left PLUS MINUS
%left MUL DIV

%%
expr : expr PLUS expr   { $$ = $1 + $3; }
     | expr MUL expr    { $$ = $1 * $3; }
     | LPAREN expr RPAREN { $$ = $2; }
     | NUM              { $$ = $1; }
     ;
%%
```

Division of work

Lex identifies NUM and operators; Yacc handles precedence and recursive expression structure.

Why not parse with Lex?

Regular expressions cannot naturally express nested arithmetic expressions with matching parentheses.

Common Lex mistakes

Most scanner bugs come from rule conflicts

Mistake	Symptom	Fix
ID before keywords	if becomes ID	Put keyword rules first or use lookup
No longest-match thinking	>= split into > and =	Define multi-char operators
Missing fallback rule	Scanner jams on bad char	Add . rule for error
Ignoring line numbers	Poor diagnostics	Update yylineno / columns

Debugging method

Print tokens for small programs before connecting the scanner to a parser.

Practice exercise: build keyword automata

Classroom activity

- Draw a combined trie automaton for: if, int, in, else, enum, for, float.
- Mark accepting states with token names.
- Add the identifier DFA and explain why float1 is ID, not FLOAT followed by NUM.
- Write equivalent Lex rules in two ways: explicit keyword rules and reserved-word lookup.

Keywords:
if int in else enum for float

Test inputs:
if in2 integer float float1

Expected insight

Keyword automata recognize exact words; identifier rules and longest-match protect prefix cases.

Summary

Lexical analysis is where automata become working compiler code

Core idea

A lexical analyzer maps characters to tokens using regular languages and finite automata.

Keyword lesson

Keywords can be recognized with automata or by matching identifiers and consulting a reserved-word table.

Tool lesson

Lex writes the scanner; Yacc writes the parser; token names and values form their interface.

Final mantra: longest match first, rule order second, attributes through yylval.